

Decentralized Computing and Blockchains

Ethereum

Alysson Bessani
DI-FCUL

Preface: Bitcoin Programming

- Bitcoin supports a simple scripting language to program transactions
- This language, called Script, assumes a stack-based machine (see <https://en.bitcoin.it/wiki/Script>)
 - The language is quite limited: no loops, simple operations for validating transactions

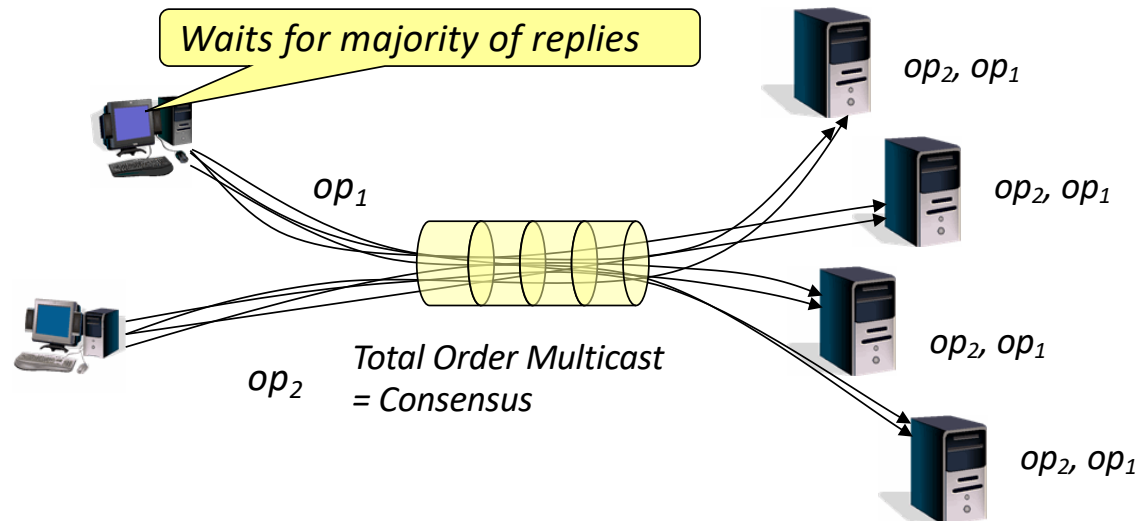
```
scriptPubKey: <expiry time> OP_CHECKLOCKTIMEVERIFY OP_DROP OP_DUP OP_HASH160  
<pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG
```

```
scriptSig: <sig> <pubKey>
```

Stack	Script	Description
Empty.	<sig> <pubKey> OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG	scriptSig and scriptPubKey are combined.
<sig> <pubKey>	OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG	Constants are added to the stack.
<sig> <pubKey> <pubKey>	OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG	Top stack item is duplicated.
<sig> <pubKey> <pubHashA>	<pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG	Top stack item is hashed.
<sig> <pubKey> <pubHashA> <pubKeyHash>	OP_EQUALVERIFY OP_CHECKSIG	Constant added.
<sig> <pubKey>	OP_CHECKSIG	Equality is checked between the top two stack items.
true	Empty.	Signature is checked for top two stack items.

Preface: State Machine Replication

- Consider a server accessed by several clients
 - This server provides some service such as file storage, naming & directory, coordination, database, etc.
- How to make it Byzantine fault-tolerant?





- Ethereum is a decentralized, open-source blockchain with a built-in Turing-complete language (and execution environment)
- [2013] Proposed by Vitalik Buterin
- [2014] Crowdfunding launched
- [2015] Live network
- [2016] Hard Fork (spanning Ethereum Classic)
- [2022] Ethereum 2.0 (Proof-of-Stake)

We focus on general aspects of Ethereum unless noted otherwise



- Ethereum allows the creation of self-enforcing smart contracts to control any asset-transfer interaction.
- Contracts are state machines replicated on the network.
 - An **Internet Computer**!
- Ethereum's crypto-currency is called **Ether**.
- **Ether** serves two purposes:
 - A **value** representation to be transferred.
 - An **incentive** mechanism for controlling the execution of smart contracts.



Ethereum vs. Bitcoin

(Technical Aspects)

- In Ethereum, the block time is set to approximately 12 seconds (Bitcoin uses 10 minutes).
 - This allows for faster transaction times (higher throughput and bigger blocks).
 - Ethereum 1 does this by using ideas from GHOST (a variant of Nakamoto consensus)
- Contrary to Bitcoin, Ethereum has its own Turing-complete instruction set, i.e., scripts can compute anything.
- While Bitcoin transactions compete equally with each other, Ethereum has a different method for costing transactions.
 - The transaction cost depends on its computational complexity, size, and storage needs.
- Ethereum 1.0 (and EthereumClassic and EthereumPoW) uses a memory-hard hashing algorithm called ETHASH (instead of SHA256)
 - Mitigates the use of ASICS (specialized hardware) and encourages decentralized mining by individuals using GPUs.

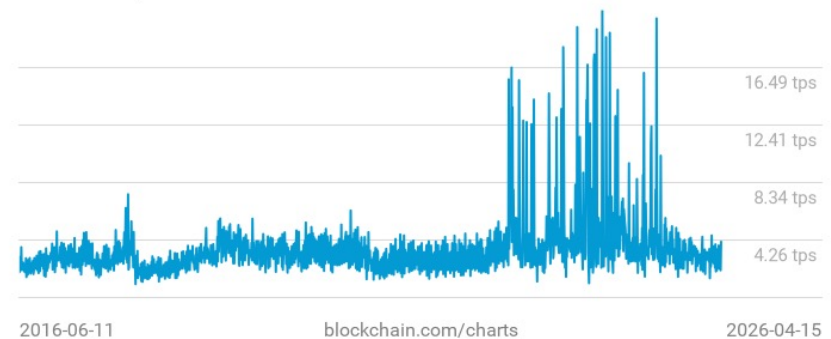
(From a previous lecture)

Some numbers about the Bitcoin network

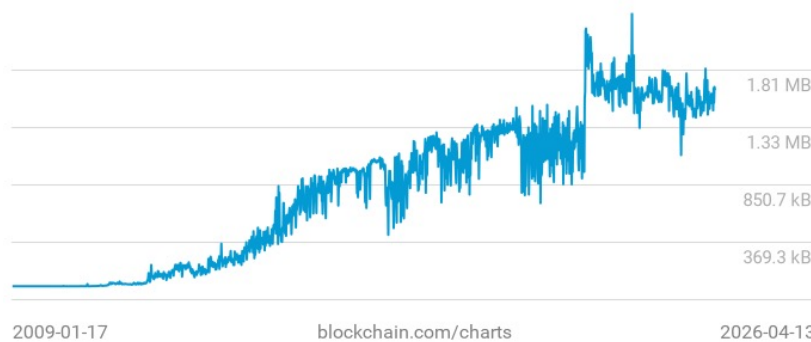
Median Confirmation Time
12.5 min



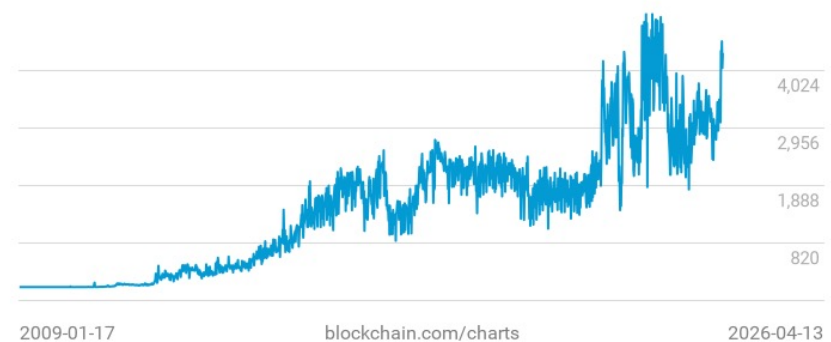
Transaction Rate
4.13 tps



Average Block Size
1.65 MB



Average Number Of Transactions Per Block
4,331



Some numbers about the Ethereum network

Ethereum Average Block Size Chart

Source: Etherscan.io

Click and drag in the plot area to zoom in



DCB

Ethereum Daily Transactions Chart

Source: Etherscan.io

Click and drag in the plot area to zoom in



8

Ethereum Execution Flow

1. A mining/validator node selects a list of pending transactions from the transaction pool and executes them
 - Including the deployment and invocation of smart contracts
2. Results from these executions will change the state of the system
 - Accounts and/or smart contracts
3. Transactions and updated data are packed into a block with the hash of the previous block, plus a nonce proving the PoW (Ethereum 1.0) is solved
 - The generated block is broadcasted to other peers using the P2P gossip network
4. Each node that receives the block verifies its header (including the PoW, in Ethereum 1.0) and executes the transactions (i.e., smart contract code) in the order specified in the block, and verifies if the recorded state on the block is correct
5. Finally, receiving nodes record the resulting changes to account balances and data stored with smart contract code, and add the block to the their own blockchain

Ethereum State

- Ethereum's state is made of objects called accounts
- Each account is identified by a 20-byte address
- Two types of accounts:
 - Externally-owned accounts, controlled by private keys
 - Contract accounts, controlled by their code
- An account contains four fields:
 - A nonce counter used to ensure each transaction issued by this account is processed at most once
 - The account Ether balance
 - The account contract code, if present
 - The account storage (initially empty)
- Ethereum state is changed by executing transactions

Bitcoin Blockchain

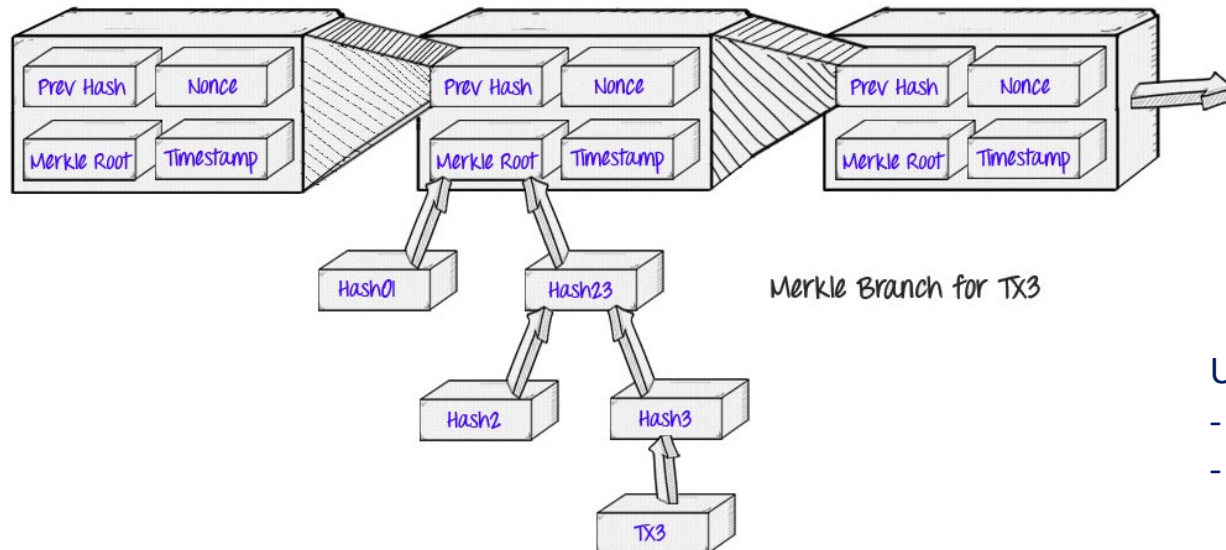
(Headers vs. Full Blockchain in Bitcoin)

- Bitcoin' blockchain size (May 2017, ~468k blocks):

- Full blockchain: 138GB
- Blockchain headers: 37MB

Update (May 2019):
- 578k blocks
- 259 GB

Update (March 2021):
- 671k blocks
- 328 GB

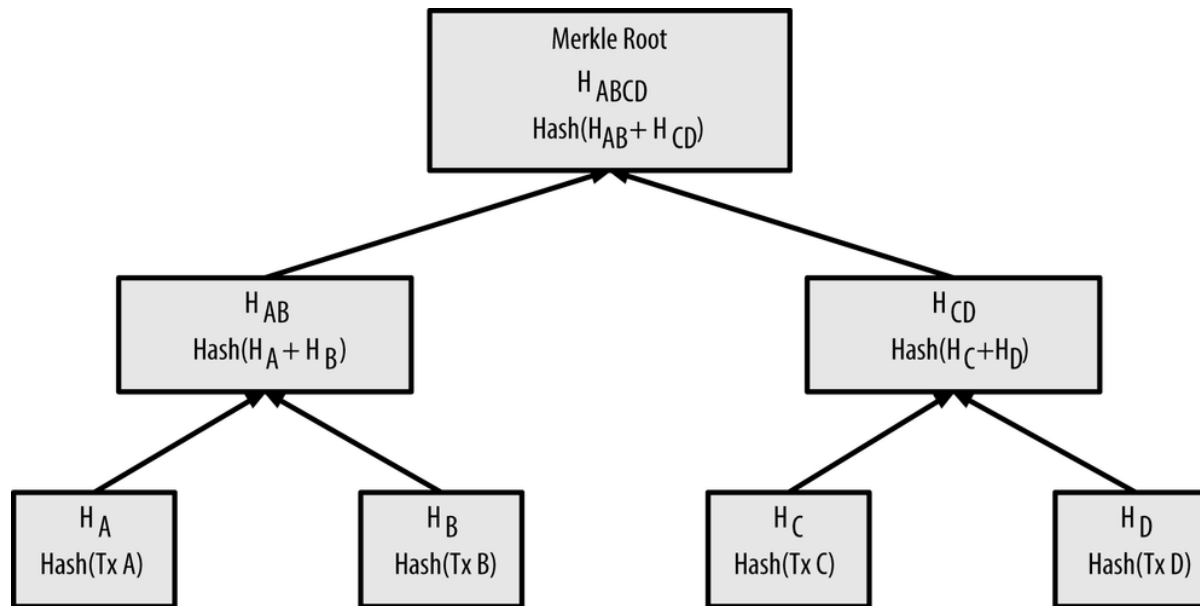


Update (April 2023):
- 787k blocks
- 476 GB

Update (April 2025):
- 895k blocks
- 649 GB

Merkle Trees

- Merkle trees are used to store, prove and validate that a transaction belongs to a block

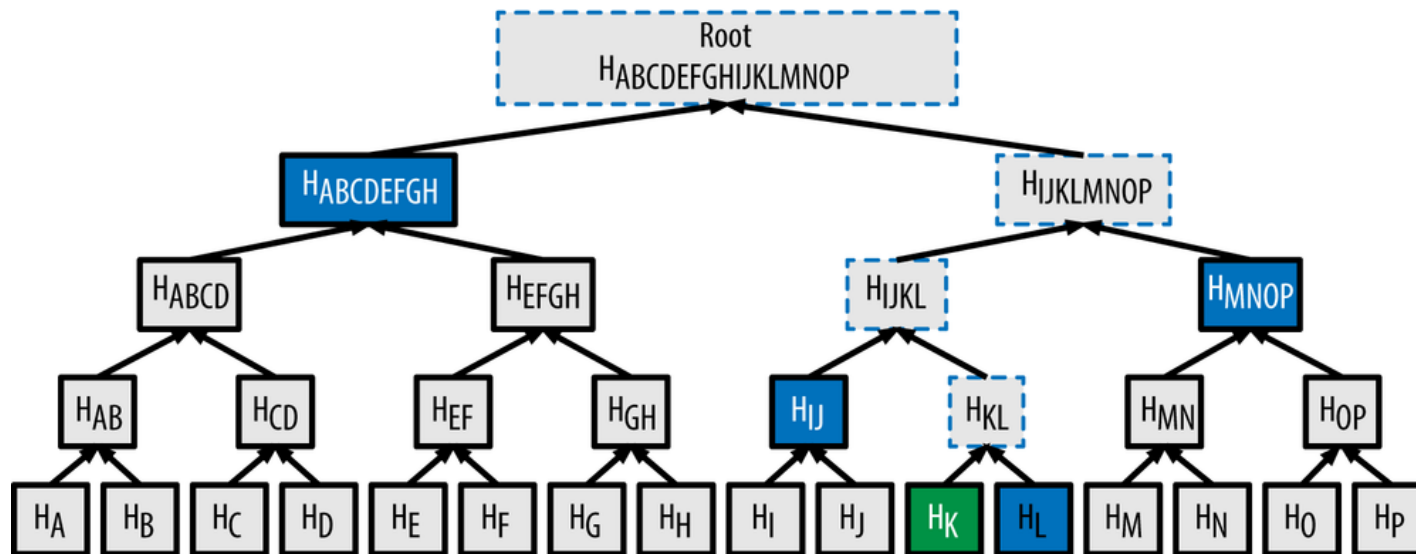


[<https://www.oreilly.com/library/view/mastering-bitcoin/9781491902639/ch07.html>]

Merkle Trees

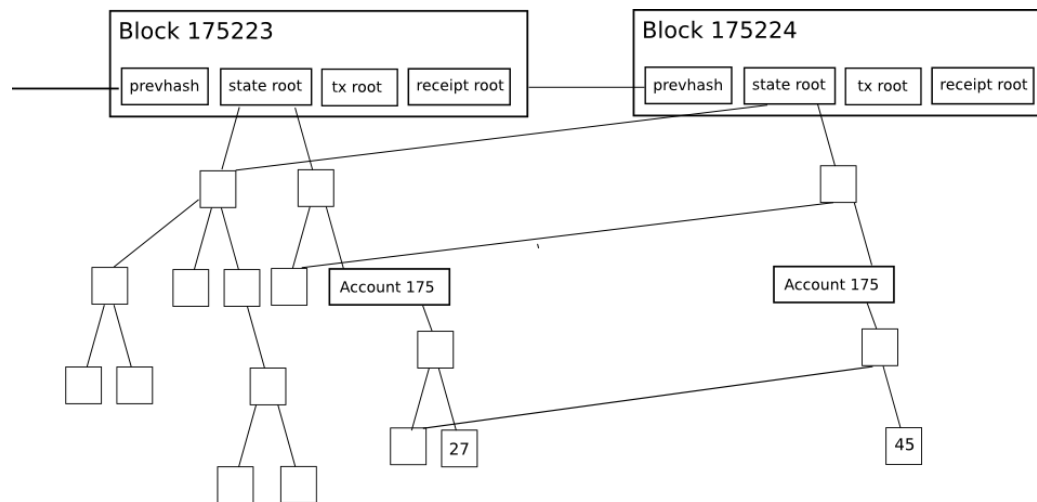
- A proof takes a path with 4 steps here = 128 bytes (4×32)
- Validating that H_K belongs to the block requires calculating only 4 pair-wise hashes

[<https://www.oreilly.com/library/view/mastering-bitcoin/9781491902639/ch07.html>]



Ethereum Blockchain

- Ethereum's block structure is like Bitcoin's, but besides the transactions tree it also contains the results of the transactions execution and state changes
- Size (May 2017, 3765k blocks): Full 51.4GB, Light ~2GB



[<https://blog.ethereum.org/2015/11/15/merkling-in-ethereum/>]

Update
(April 2023):

- 17132k blocks
- Full 934 GB

Update (April 2025):

- 22487k blocks
- 1268 GB

Blocks per day.
Sep'23: Ethereum 2
was deployed.



Ethereum Transactions

- An Ethereum transaction (called *Message*) has three important differences when compared with Bitcoin's:
 - Can be created either by an external entity or by a smart contract
 - Ethereum messages can contain data
 - The recipient of a message can send a response
- An Ethereum message contains:
 - The recipient of the message
 - A signature identifying the sender
 - The amount of ETH to send
 - The data to send
 - Two values called STARTGAS and GASPRICE
- Executing a smart contract uses gas. STARTGAS and GASPRICE defines the resources the message sender can commit for transaction execution

Ethereum Transaction Execution

1. Check if the transaction/message is well formed, signature is valid, and nonce match the sender's account nonce
2. Calculate the transaction fee as $\text{STARTGAS} * \text{GASPRICE}$, subtract the fee from the sender's account balance and increase the sender's nonce
3. Initialize gas = STARTGAS and take off a certain quantity of gas due to the size of the transaction
4. If the receiver is a contract, run the contract's code until its end or the gas is over
 - Otherwise (if the receiver is an external account), transfer the message value (Ether) to the receiver account
5. If the transfer fails because the sender did not have enough money, or the code runs out of gas, revert all the state except step 2
6. Refund the fees (if available) for all remaining gas to the sender and give the transfer fee to the miner of the transaction block

Ethereum 2.0

Gaspar, the alternative to Proof-of-Work

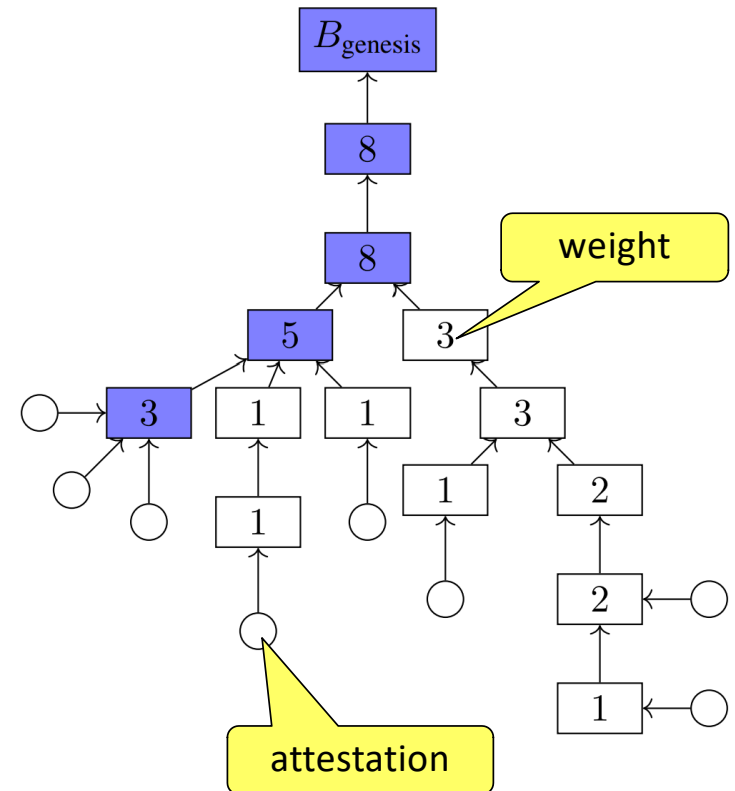


Gaspar Consensus Protocol

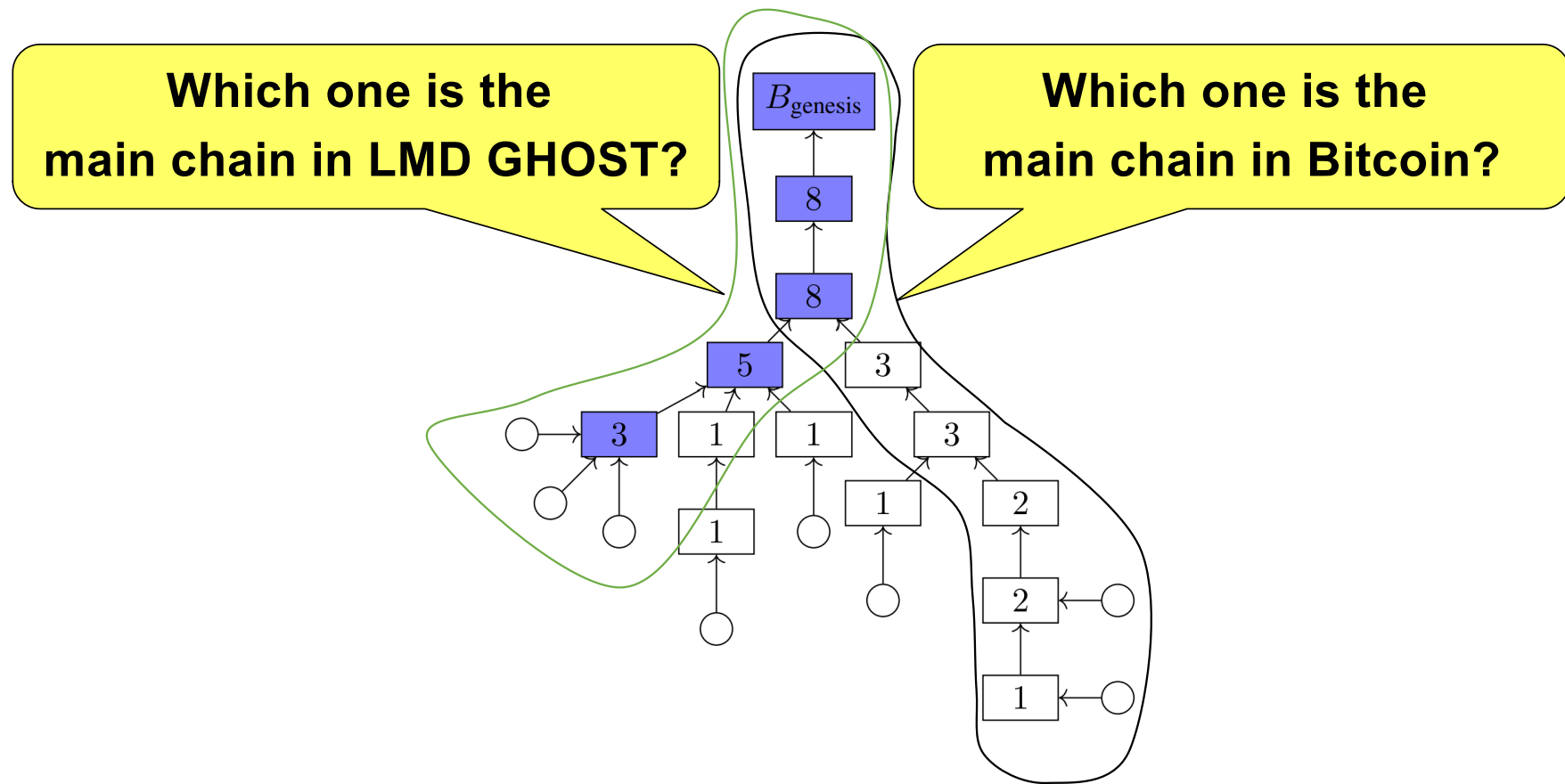
- Gaspar is a Proof-of-Stake (PoS) consensus protocol
- In a PoS protocol, miners commit cryptocurrency, not CPU, to produce blocks
- The main reasons to migrate from Nakamoto consensus (PoW, Ethereum 1.0) to Gaspar (PoS, Ethereum 2.0) are the energy spending and performance
- In Gaspar, instead of using CPU for creating a PoW for new blocks, a participant pays to participate in block generation/validation
 - A block is valid, if it contains (instead of a PoW) a certificate with $>2/3$ of validators' signatures
- Gaspar is based based on two sub-protocols (Ghost + Casper):
 - The Latest Message Driven Greediest Heaviest-Observed Sub-Tree (LMD GHOST)
 - It is a **fork-choice rule**
 - Casper the Friendly Finality Gadget (Casper FFG)
 - It is used to mark certain blocks as **finalized** so that users know if a block belongs to the main chain or not.

The LMD GHOST Choice Rule

- A strategy for selecting a chain in case of forks
- How it works:
 - Each produced block might have one or more attestations (validator signatures).
 - Each block has a weight equals to the total of its attestations and children's attestations.
 - The main chain will not be the longest (as in Bitcoin or Streamlet), but the heaviest one!
- This makes the protocol security independent of the number of forks.



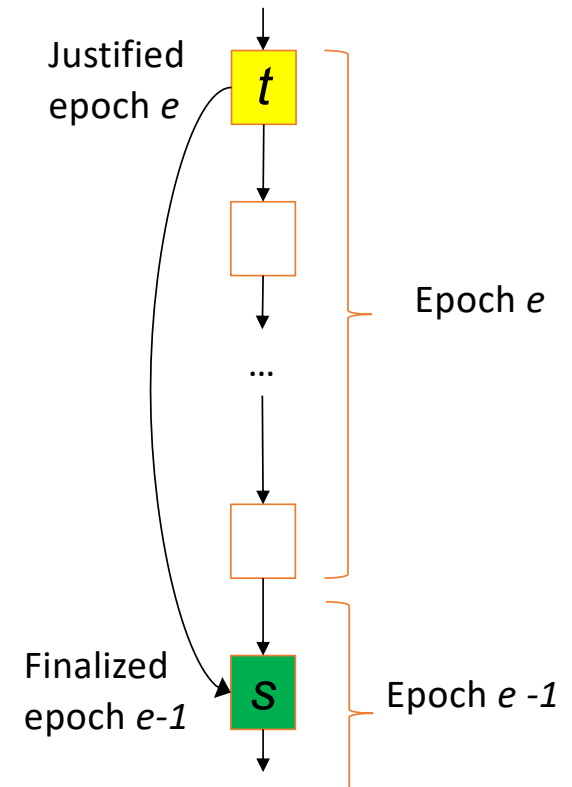
The LMD GHOST Choice Rule



Casper Finality Gadget

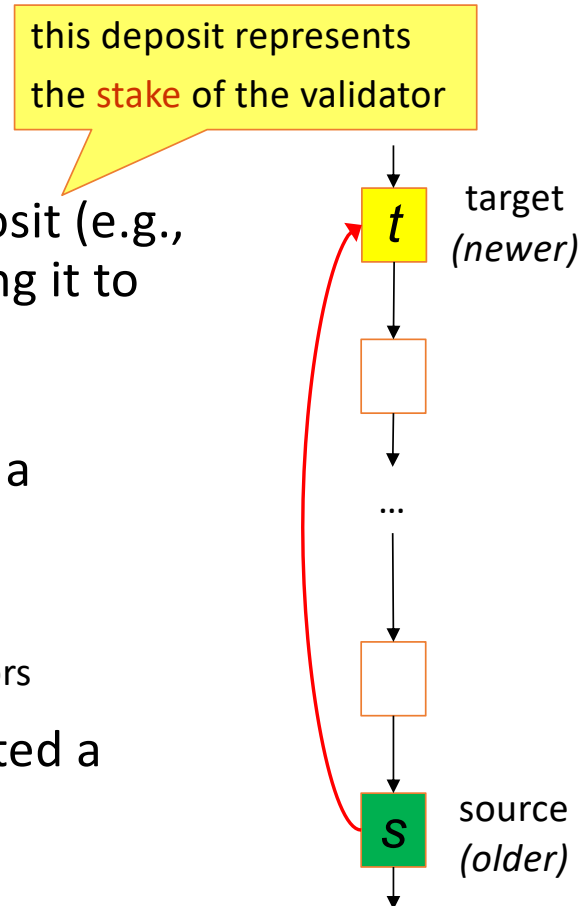


- Casper's task is to finalize/commit blocks using a committee of validators
 - Finalization means the transactions on the block and previous blocks can not be reverted!
- Key rules:
 - A block is justified when it receives enough votes from the validators committee;
 - A block (checkpoint) needs to be justified at every C (32) blocks (an epoch);
 - A block is finalized when a block for the next epoch is justified.



Casper FFG Details I

- A node can be a validator if it makes an Ether deposit (e.g., 32 Eth -> approximately €64k in April'26) by sending it to the Casper smart contract
 - This solves the “Nothing at Stake” problem
- Validators cast votes **weighted** by their stakes, i.e., a validator's vote is proportional to its stake
- Validators can vote on checkpoint blocks
 - A vote $\langle v, s, t, h(s), h(t) \rangle_{\sigma_i}$ is broadcasted to all other validators
- When more than 2/3 of the stake of validators casted a vote with s as source and t as target, we call this a **supermajority link** $s \rightarrow t$



Casper FFG Details II

- Justified checkpoint

A checkpoint t is **justified** if it is the genesis block or if there is a supermajority link $s \rightarrow t$ where s is justified.

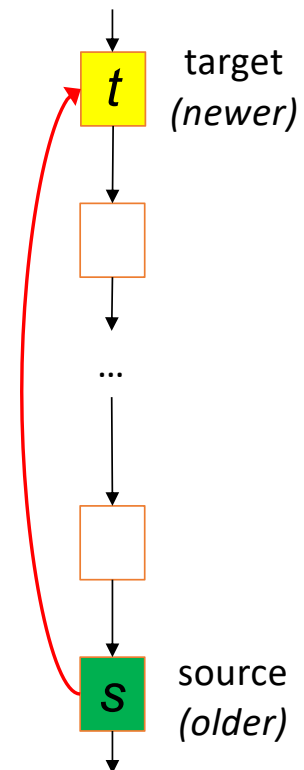
- Finalized checkpoint

A checkpoint s is **finalized** if it is justified and there is a supermajority link $s \rightarrow t$ where t is a direct child of s .

- Rewards and penalties:

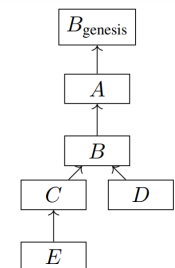
- When a validator votes on a finalized checkpoint, it gets some reward
- When a validator votes on a checkpoint that is never finalized, it loses some money
- If a validator misbehaves, it might lose its deposit

<https://ethereum.org/developers/docs/consensus-mechanisms/pos/rewards-and-penalties/>



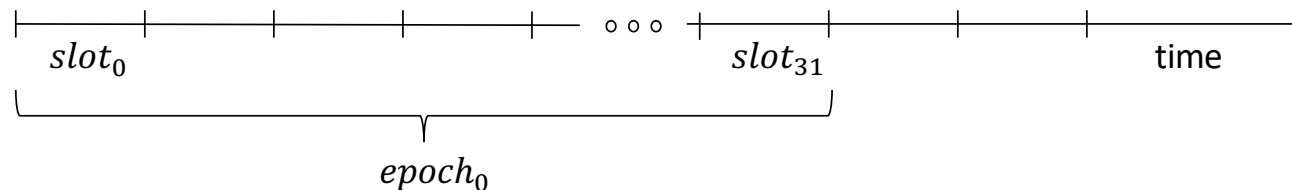
The Ethereum 2 (Gasper) Protocol: Model

- The consensus participants are called validators $\mathcal{V} = \{v_1, v_2, \dots, v_N\}$.
- Each validator v has an amount of stake $w(v)$ s.t. $\sum_{v \in \mathcal{V}} w(v) = \text{total stake}$
- A validator can be honest or Byzantine.
- Broadcast is the only communication primitive by which honest validators can communicate with each other, i.e., an honest validator never send a message to a specific subset of validators.
- Each validator v has its own view of the system at any time t denoted by $view(v, t)$:
 - $view(v, t)$ contains the set of all blocks that v has seen until t .
 - These blocks form a tree rooted at a genesis block $B_{genesis}$.



The Ethereum 2 (Gasper) Protocol: Model

- The system is partially synchronous, and time triggered (sync. clocks)
 - Slot: some constant number of seconds (e.g., 12 seconds)
 - Epoch: some constant number of slots C (e.g., 32 slots – 6.4 minutes)
- In each slot, at most one block can be added to the blockchain
- Blocks belonging to epoch j have slot numbers $jC+k$, where $0 \leq k \leq C-1$
- $B_{genesis}$ has slot number 0 and is the first block of epoch 0



The Ethereum 2 (Gasper) Protocol

// The pseudocode for validator v
 // $\mathcal{V}$: the set of validators
 // C : the number of slots per epoch

Function Main()

Fork Attestation()

$epoch \leftarrow 0$

Loop

$S_0, S_1, \dots, S_{C-1} \leftarrow \text{GetCommittees}()$

For each slot $i \in \{epoch \times C, \dots, (epoch + 1) \times C - 1\}$

If $v \in S_i$

// The first half of slot i

If $v = S_i[0]$ // v is the leader

$B \leftarrow \text{GetBlock}()$

$P(B) \leftarrow \text{LMD_GHOST}(\text{view}(v, i))$ // $P(B)$ is the parent of B

Broadcast $\langle \text{PROPOSE}, B, i, P(B) \rangle_{\sigma_v}$ to $\mathcal{V}$

// The second half of slot i

If there is a message $\langle \text{PROPOSE}, B, i, P(B) \rangle_{\sigma_u}$

If u is the leader $\wedge B$ is valid $\wedge \text{LMD_GHOST}(\text{view}(v, i)) = P(B)$

Add B to the local view

$S \leftarrow$ the last slot of the last justified epoch

Broadcast $\langle \text{ATTESTATION}, B, i, P(B), S \rangle_{\sigma_v}$ to $\mathcal{V}$

$epoch \leftarrow epoch + 1$

Next slide

Function GetCommittees()

$\pi \leftarrow$ get a random permutation of $\mathcal{V}$

$\mathcal{S} \leftarrow \emptyset$

For $i \in \{0 \dots C - 1\}$

$S_i \leftarrow \pi[i \times \frac{|\mathcal{V}|}{C} : (i + 1) \times \frac{|\mathcal{V}|}{C} - 1]$

$\mathcal{S} \leftarrow \mathcal{S} \cup \{S_i\}$

Return $\mathcal{S}$

Creates C committees

Each committee is responsible for a single slot

The first committee member is the leader

Each slot has two parts:

- In the first half, a block is proposed by the leader of the committee
- In the second half, validators of a committee attest to the proposed block.

The Ethereum 2 (Gasper) Protocol

Function `Attestation()`

 Loop

 Let A be the set of received attestations from finished epochs

 Let $A[j]$ be the set of attestation from epoch j

 // 2/3 majority rule for justification

 If $\exists j : \sum_{a \in A[j]} \text{GetValidatorStake}(a) \geq \frac{2}{3} \times \sum_{u \in \mathcal{V}} \text{GetStake}(u)$

$S \leftarrow$ the last slot of the last justified epoch

 If blocks created during epoch j point to S

 Mark j as justified

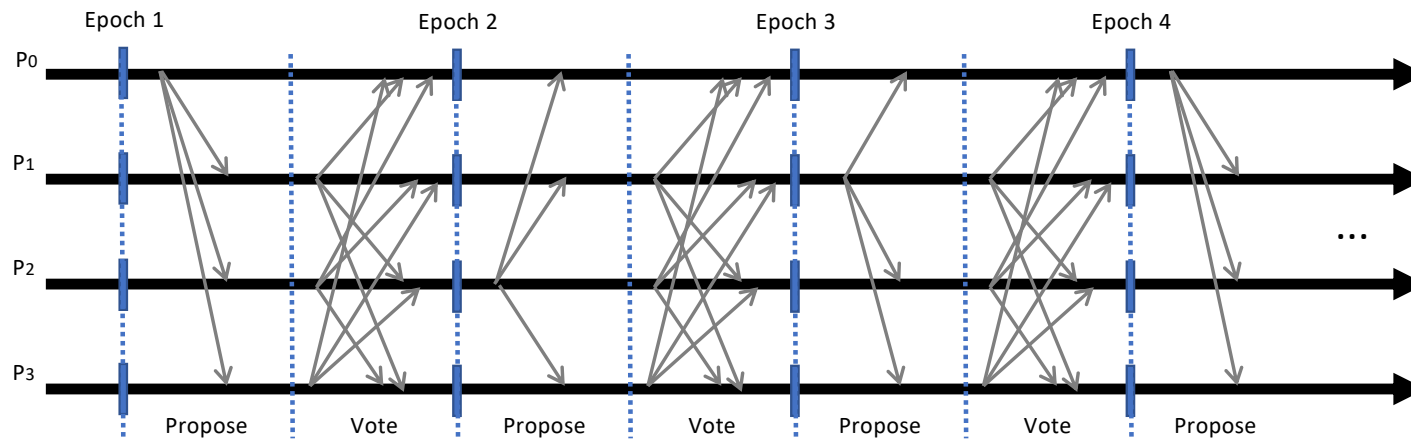
 If S was in epoch $j - 1$

 Mark $j - 1$ as finalized

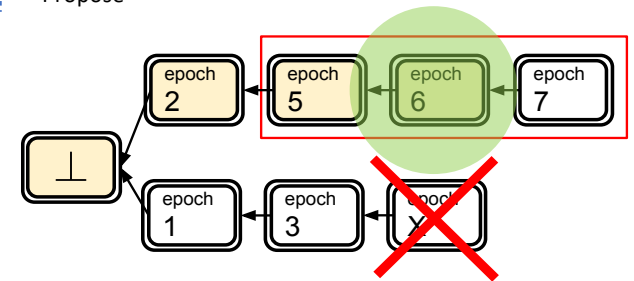
This is a separate thread that receives attestation and mark votes as justified and finalized when the conditions defined on Casper are satisfied.

(From a previous lecture)

Comparing Gasper with Streamlet



- Epoch leader proposes its block
- Each node that accepts this block, votes for it with its signature
- A block for epoch e is finalized (committed) if blocks for epochs $e-1$, e , $e+1$ are notarized



Comparing Gasper with Streamlet

- If we ignore the specific details, Gasper is quite similar to Streamlet
- The epochs are similar, but in Streamlet, there is only one block proposal, and all attestations are done in parallel.
- In both algorithms, in the best case, at every two epochs, one block (or block sequence) is finalized.
- The notion of committees and time partitioning (in slots) is used in Gasper for improved scalability.
- Both protocols tolerate asynchrony, losing liveness.
- The average finality time in Ethereum is 14 minutes.

(some) Bibliography

- V. Buterin. *Ethereum White Paper* (A Next Generation Smart Contract & Decentralized Application Platform)
- G. Wood. *Ethereum Yellow Paper (Ethereum: a Secure Decentralised Generalised Transaction Ledger)*.
- Y. Sompolinky, A. Zohar. *Secure High-Rate Transaction Processing in Bitcoin*. Financial Crypto. 2015.
- V. Buterin et al. *Combining GHOST and Casper*. <https://arxiv.org/abs/2003.03052>. 2020.